CSC 3320: System-Level Programming

Chapters 8 and 9

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Note: These slides are updated based on original notes from Raj Sunderraman, Michael Weeks, and Yuan Long

Chapter 8: Arrays
Chapter 9: Functions

What will be covered?

- How to define arrays?
- How to trace arrays?
- How to define functions?
- How to declare functions?
- How to call functions?
- Arguments vs. Parameters
- Program termination
- Recursive function

C programming---Arrays

- **scalar**: capable of holding a single data item
- **aggregate variables**: capable of holding a collections of values.
  - Arrays
    - An **array** is a data structure containing a number of data values, all of which have **the same type**.
  - Structures (records)
    - Will be covered in Chapter 16

Arrays

- One-Dimensional Arrays
  - The simplest kind of arrays
  - The elements, the values of the items, of a one-dimensional array are conceptually arranged one after another in a single row.

  - Declare an array

e.g. `int a[8];`

In Java we should write
`int[] a=new int[8];`

e.g.
```
#define N 10
……
int a[N];
```

N will be replaced by 10 when program is compiled

Array Subscripting

- The elements of an array of length n are indexed from 0 to n-1

```
a[1] = 9;
printf("%d\n", a[5]);
++a[i];
```

Arrays and For Loop

- Arrays and *for* loops go hand-in-hand
- Idioms:

```
for(i = 0; i < N; i++)
  a[i]= 0;
```

Clears a

```
for(i = 0; i < N; i++)
  scanf("%d", &a[i]);
```

Reads data into a

```
for(i = 0; i < N; i++)
  sum += a[i];
```

Sums the elements of a

Example: taking inputs into array

```
#include<stdio.h>
#include<stdlib.h>


int main()
{
    int myarray[10];
    printf("Please provide 10 inputs:");
    for(int i=0; i<10; i++)
       scanf("%d", &myarray[i]);
    return 0;
}
```

Example: random number

```c
#include<stdio.h>
#include<stdlib.h>

int main()
{
    srand(0);
    int random_number;

    // use % if wanted in a range, e.g. 0-14
    random_number = rand() % 15;

    printf("%d ", random_number);
    printf("\n");

    return 0;
}
```

Arrays and For Loop

- Note: Be careful for its indexing
  - C doesn't require that **subscript bounds** be checked.
  - If a subscript goes out of range, the program's behavior is undefined.
    - No compiling error but the result may be unexpected.

```
int a[10], i;
for(i = 1; i <= 10; i++)
  a[i] = 0;
```

Note: actually a[10] does not exist. With some compilers, this for statement causes an infinite loop!

Array subscript may be any integer expression

```
a[i+j*10]
```

```
i = 0;
while(i < N)
   a[i++] = 0;
```

Is a[++i] right?

```
i = 0;
while(i < N)
   a[i] = b[i++];
```

May cause side effect. How to avoid this problem?

```
for ( i = 0, i < N, i++)
  a[i] = b[i];
```

Array Initialization

- Always initialize array when you declare it

```
int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};


int a[10] = {1 , 2, 3, 4, 5, 6};
    /* {1, 2, 3, 4, 5, 6, 0, 0, 0, 0} */

int a[10] = {0};
    /* {0, 0, 0, 0, 0, 0, 0, 0, 0, 0} */

int a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    /* {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} */
```

Using the *sizeof* Operator with arrays

- The **sizeof** operator can determine the size of an array (in bytes).
  - Example
    - int a[10]

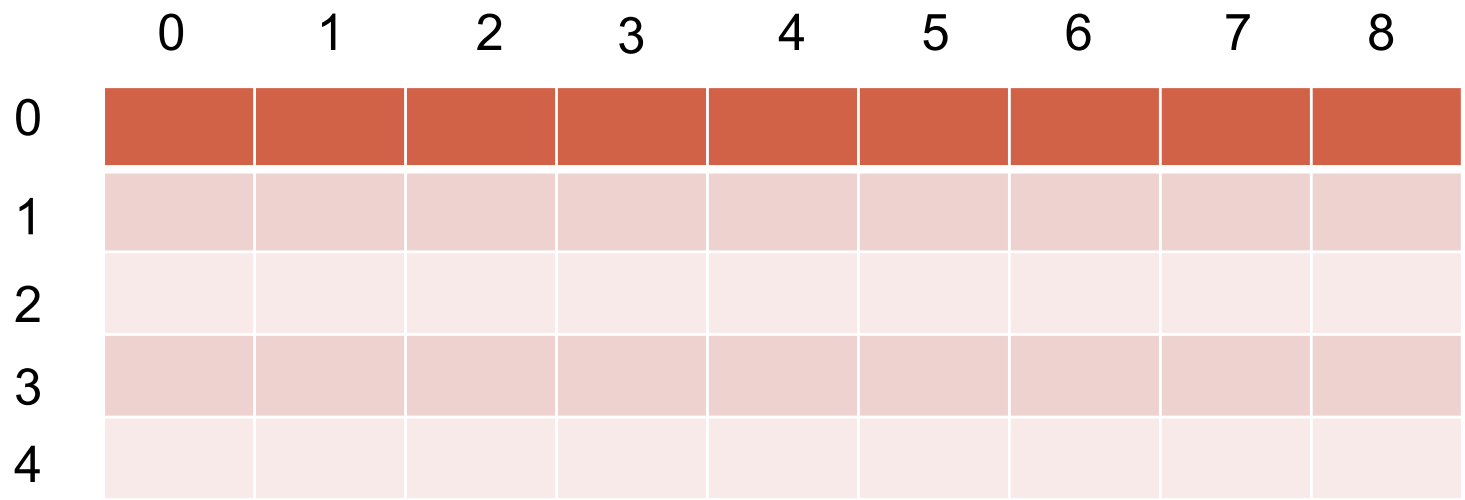
    sizeof(a) returns 40 (assuming each integer requires 4 bytes)
- Measure the size of an array
  - `sizeof(a) / sizeof(a[0]);`
  - Example:

```
#define SIZE ((int) (sizeof(a) / sizeof(a[0])))

for (i = 0; i< SIZE ; i++)
    a[i] = 0;
```

Multidimensional Arrays

- int m[5][9]; // 5 rows 9 columns

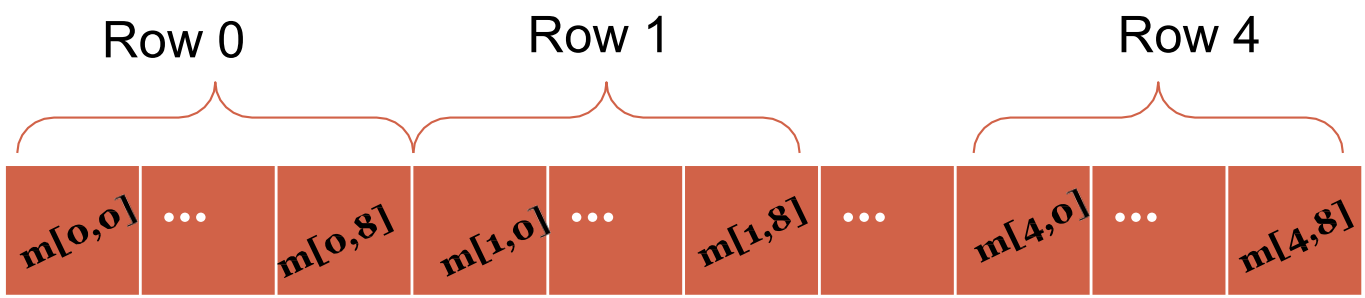


In Java we should write

```
int[][]  m=new int[5][9];
```

Multidimensional Arrays

- How multidimensional array stored in Memory?



- Example: Initialize an array for use as an identity matrix

```
#define N 10
void main(){
    double ident[N][N];
    int row, col;
    for(row = 0; row < N; row++)
        for(col = 0; col < N; col++)
            if(row == col)
                ident[row][col] = 1.0;
            else
                ident[row][col] = 0.0;
}
```

Multidimensional Arrays

- Initializing a Multidimensional Array

```
int m[5][9]  = { {1,1,1,1,1,1,1,1,1},
                 {0,1,0,0,0,1,1,0,1},
                 {0,0,0,0,0,0,0,0,0},
                 {1,1,1,1,1,1,1,1,1},
                 {1,0,1,0,1,0,1,0,1}}
```

```
int m[5][9]  = { {1,1,1,1,1,1,1,1,1},
                 {0,1,0,0,0,1,1,0,1},
                 {1,0,1,0,1,0,1,0,1}}
```

```
int m[5][9]  = { 1,1,1,1,1,1,1,1,1,
                 0,1,0,0,0,1,1,0,1,
                 0,0,0,0,0,0,0,0,0,
                 1,1,1,1,1,1,1,1,1,
                 1,0,1,0,1,0,1,0,1}
```

```
int m[5][9] = { {1,1,1,1,1,1,1,1,1},
                {0,1,0,0,0,1,1,0},
                {0,0,0,0,0,0,0,0},
                {1,1,1,1,1,1,1},
                {1,0,1,0,1,0,1,0,1}}
```

Functions

- Similar to Java methods
- Functions are defined in a C file, instead of a class.
  - Each function is essentially a small program, with its own declarations and statements.
  - Using functions, programs are divided into small pieces that are easier for us as well as others to understand and modify.
  - Functions can take some of the tedium out of programming by allowing us to avoid duplicating code that's used more than once.

Defining and calling functions

- Example of defining a function

```
double average(double a, double b)
{
        return (a + b) / 2;
}
```

- Example of calling a function

```
avg = average(x/2, y*2);
printf("Average: %g\n", average(x, y));
```

Function Definitions

```
return-type   function-name ( parameters )
{
    declarations
    statements
}
```

- Note
  - Function may not return arrays.
  - Specifying that the return type is ***void*** indicates that the function doesn't return a value
  - If the return type is omitted, the function is presumed to return a value of type **`int`**

```
double average(double a, b)      /** WRONG **/
```

Issues with increment operator

```
#include<stdio.h>

int main()
{
   int a[5], b[5]={1, 2, 3, 4, 5};
   int i = 0;

   while (i < 5)
   {
      a[i] = b[i++] ; // Be very careful, it may produce ambiguous results
      printf("a[%d]=%d\t", i, a[i]);
   }

   printf("\nCopied array is:");
   for(i=0; i<5; i++)
      printf("%4d", a[i]);

   printf("\n");
   return 0;
}
```

```
Output:

a[1]=1 a[2]=2 a[3]=3 a[4]=4 a[5]=5
Copied array is:4195904   1   2   3   4
```